

skillctl Command Reference

Every subcommand, every flag, every exit code

m3c · Scalytics — KuP Berlin training

2026-05-07

Contents

skillctl – Command Reference Manual	2
Conventions	2
Top-level usage	4
Authoring (signing, packing, verifying)	5
pack – build a deterministic .skb (G)	5
keygen – generate an ed25519 keypair (G)	5
sign – sign a .skb bundle (G)	6
verify-sig – verify a detached author signature locally (G)	6
Trust-root management	7
trust list – show pinned registries (G)	7
trust add – pin a registry (G)	7
trust remove – unpin a registry (G)	7
Inventory	8
scan – walk skill directories (G)	8
report – render a scan as HTML or markdown (G)	8
diff – compare two scans (G)	9
seal – freeze a scan as baseline (G)	9
import – push a scan to a remote target (G)	9
audit – antivirus-shaped scanner (G) (Phase 2 implementation underway)	9
review – open the local TUI for a delta (G)	10
browse – read-only inventory TUI (G)	10
consolidate – fix annotation gaps in a baseline (G)	10
sync-usage – push local skill-usage telemetry (G)	10
Awareness bridge (SPEC-0195)	11
awareness sync – admit local skills to a registry (G)	11
awareness verify – read back per-session admissions (G)	11
awareness reset – delete a session’s admissions (G)	12
Intent declaration (SPEC-0196)	12
intent declare – declare or update intent + data_dependencies (G)	12
Author ingestion (SPEC-0194)	13
propose – ready-to-promote gate + pack + sign + push (Y) [drafted]	13
Verifier (SPEC-0188 §7, §11)	13

install – pull, verify, install a signed skill bundle (G)	13
verify – re-run the trust-chain check on an installed skill (G)	14
Governance attestation	14
attest – POST a signed governance attestation (G)	14
Upstream airlock (SPEC-0201)	15
import-public – stage an upstream bundle (Y) [drafted]	15
import-list – list staged imports (Y) [drafted]	15
import-policy lint – validate the source policy (Y) [drafted]	16
Runtime envelope (SPEC-0202)	16
run – wrap an invocation under a capability token (Y) [drafted]	16
invoke-replay – re-run a recorded invocation (Y) [drafted]	17
Author-key revocation (SPEC-0198)	17
Conventions in command output	17
Notable cross-command flags	17
Worked examples (smallest viable runs)	18
E1 – Sign and verify a bundle locally (no registry)	18
E2 – Pin a registry, install, run	18
E3 – SKILLOR-style awareness sync	18
E4 – Reviewer attestation	19
E5 – Import from a public hub (drafted SPEC-0201)	19
What this manual does NOT cover	20
References	20

skillctl – Command Reference Manual

This is the comprehensive command reference for skillctl, the Go CLI front-end for the m3c Skill-Manager. It extends [SKILLOR-WORK/s1/USER-MANUAL.md](#) (which covered only the awareness-sync subset) with the full surface area: every subcommand from the s2/integration branch's cmd/skillctl/main.go dispatcher, plus the SPEC-0194/-0196/-0201/-0202 commands that are drafted but not all wired yet.

For role-keyed playbooks (when to run what, in what order, as which role), use [USER-MANUAL.md](#).

Conventions

Notation	Meaning
<arg>	Required positional argument
[--flag VALUE]	Optional flag
--flag VALUE \ OTHER	Pick one of the values
[shipped]	Wired in s2/integration, runnable today
[drafted]	SPEC published, code in flight or staged
[planned]	Interface frozen in SPEC, no code yet

Status legend:

- (G) = green-track – skillctl pack, keygen, sign, verify-sig, attest, trust, install, verify, scan, report, diff, seal, import, audit, review, browse, consolidate, sync-usage, intent, awareness sync|verify|reset
- (Y) = drafted – propose (SPEC-0194), import-public/import-list/import-policy lint (SPEC-0201), run/invoke-replay (SPEC-0202)
- (R) = planned – KafScale-coupled fan-out commands (SPEC-0193)

The dispatcher's hardcoded set today: pack | keygen | sign | verify-sig | trust | attest | install | verify | scan | report | diff | seal | import | audit | review | browse | consolidate | sync-usage | intent | awareness | help. Drafted commands listed below as [drafted] are documented at the planned interface; verify with skillctl <cmd> --help before scripting against them.

Default key/file locations:

Path	Contents	Mode
~/.claude/skillctl-keys/author.{priv,pub}	Author signing keypair (PEM PKCS#8 / SPKI ed25519)	0600 / 0644
~/.claude/skillctl-keys/reviewer.{priv,pub}	Reviewer signing keypair (post-Q3-2026 personal-key path)	0600 / 0644
~/.claude/skill-trust-roots.yaml	Pinned registry public keys + governance minima + tenant scope	0644
~/.claude/skill-import-policy.yaml	SPEC-0201 source policy	0644
~/.cache/m3c/imports/<sha256>	Staged upstream imports (post-airlock, pre-propose)	0700
~/.config/m3c/skill-keys/	Suggested location for keygen --out	0700
~/.m3c-tools/skill-usage.db	SQLite skill-usage telemetry (syncd via sync-usage)	0600

Environment variables:

Variable	What	Used by
M3C_REGISTRY_URL	Default registry base URL (override per-command via --registry)	awareness, attest, install, verify, etc.
SKILLCTL_BIN	Path to a non-PATH skillctl binary	wrappers (e.g. tools.skill_awareness.run)
SKILLCTL_DEV_SEED	Use deterministic dev keypair seeded from this string (DEV ONLY)	awareness sync (test/dev)
M3C_SKILL_CAPABILITY_TOKEN	Live capability token (set by skillctl run; consumed by gateway shim)	runtime envelope (SPEC-0202)
GOOGLE_APPLICATION_CREDENTIALS	Filestore path	server-side flows

Exit codes: documented per-command below; the centralised catalog is in [USER-MANUAL.md §8](#).

Top-level usage

Usage: skillctl <command> [args]

Commands (Stream S1 / SPEC-0188 Phase 2):

keygen	Generate an ed25519 keypair (PEM, PKCS#8 / SPKI).
sign	Sign a .skb bundle with an ed25519 private key.
verify-sig	Verify a detached author signature locally.
attest	POST a signed governance attestation to the registry.

Commands (Stream S7 / SPEC-0188 Phase 4):

trust	Manage ~/.claude/skill-trust-roots.yaml (list/add/remove).
-------	--

Commands (Stream S8 / SPEC-0188 Phase 4):

install	Pull, verify, and install a signed skill bundle.
verify	Re-run the trust-chain check on an installed skill.

Commands (SPEC-0188 Phase 1 PoC):

pack	Build a deterministic .skb tar from a skill directory.
------	--

Commands (SPEC-0189 / inventory):

scan	Walk skill directories; emit JSON/table inventory.
report	Render a scan as HTML or markdown.
diff	Compare two scans.
seal	Freeze a baseline scan.
import	Push a scan to a remote target (--dry-run available).
audit	Antivirus-shaped scanner (verdict per skill).
review	Open a local UI to review a delta.
browse	Read-only TUI inventory browser.
consolidate	Fix annotation gaps in a baseline.
sync-usage	Sync local skill-usage telemetry.

Commands (SPEC-0195 / awareness bridge):

awareness sync	Admit local skills to a registry.
awareness verify	Read back per-session admissions.
awareness reset	Delete a session's admissions (with dry-run-token).

Commands (SPEC-0196 / intent):

intent declare	Declare or update intent + data_dependencies for a bundle.
----------------	--

Commands (SPEC-0194 / author ingestion) [drafted]:

propose	Run ready-to-promote gate, then pack + sign + push.
---------	---

Commands (SPEC-0201 / upstream airlock) [drafted]:

import-public	Stage a hub bundle under ~/.cache/m3c/imports/<sha256>/.
import-list	List staged imports.
import-policy lint	Validate ~/.claude/skill-import-policy.yaml.

Commands (SPEC-0202 / runtime envelope) [drafted]:

run	Issue a capability token, wrap invocation, gate side-effects.
invoke-replay	Re-run a recorded invocation under old or new policy.

Run any command with --help for its flags.

Authoring (signing, packing, verifying)

pack – build a deterministic .skb (G)

Purpose. Convert a skill directory into a deterministic gzip-tar .skb whose SHA-256 is the canonical bundle_digest.

Usage.

```
skillctl pack <skill-dir> [--out <path>] [--name <name>] [--version <semver>]
```

Arguments:

- <skill-dir> – directory containing SKILL.md, bundle.json, optional scripts/, helpers/, profiles/, etc.
- --out PATH – output .skb (default: <name>-<version>.skb in the current directory).
- --name NAME – override bundle.json::name (rare).
- --version SEMVER – override bundle.json::version (rare).

Determinism rules (SPEC-0188 §3.1):

- Files in lexicographic order
- Mode 0644 (or 0755 under scripts/)
- mtime = Unix epoch (1970-01-01T00:00:00Z)
- Tar PAX format with no extended headers
- Gzip header zeroed (gzip --no-name semantics)
- No .DS_Store / .git at root
- No wrapper directory (files at tar root, per BUG-0133 resolution)

Cross-field validation (SPEC-0196 §3.3) – pack refuses with exit 18 intent_inconsistent when:

- intent.destructive=true and any data_dependency.access is read-only
- intent.network=true and no data_dependency.kind=http_endpoint
- data_dependency.access=write and intent.destructive=false
- intent is missing AND data_dependencies is non-empty (or vice versa)

Output.

```
wrote bundle.skb (sha256:cd1d5c79...)
digest: sha256:cd1d5c79...
files: 12
bytes: 20480
```

Exit codes: 0 ok · 1 generic · 2 usage · 18 intent_inconsistent

Gotchas.

- The bundle's filename embeds neither the digest nor a wrapper directory by default. Two pack runs produce identical bytes (you can cmp them).
- built_at in bundle.json is *always* Unix epoch in the packed bundle. The wall-clock build time lives in the registry attestation.

keygen – generate an ed25519 keypair (G)

Purpose. Generate a fresh ed25519 keypair for use with sign, attest, awareness sync, etc.

Usage.

```
skillctl keygen --out <path>
```

Output files.

```
<path>.priv      PEM PKCS#8, ed25519 (mode 0600)
<path>.pub       PEM SPKI,    ed25519 (mode 0644)
```

Suggested location. ~/.config/m3c/skill-keys/<name> or ~/.claude/skillctl-keys/<role> (the latter is what the rest of the CLI defaults to).

Stdout.

```
wrote <path>.priv (mode 0600)
wrote <path>.pub  (mode 0644)
fingerprint: sha256:<hex>
```

Exit codes: 0 ok · 1 generic · 2 usage

Gotchas.

- Keep the .priv file off git. The gitleaks config from /security-scan flags *.priv; pre-commit hooks should refuse it.
 - For dev/test, use SKILLCTL_DEV_SEED=<string> instead – produces deterministic keys for repeatable tests (NEVER in prod).
-

sign – sign a .skb bundle (G)

Purpose. Compute the bundle's SHA-256 digest, sign the 32 raw bytes with an ed25519 private key, write a detached signature file.

Usage.

```
skillctl sign <BUNDLE.skb> --key <PATH.priv> [--identity-id <ID>]
```

Output file.

```
<BUNDLE.skb>.<digest_hex>.author.sig    (64 raw bytes, mode 0644)
```

The signature filename embeds the digest so multiple bundles sharing a directory don't collide.

Stdout.

```
digest: sha256:<hex>
identity: <ID>
fingerprint: sha256:<key-fingerprint-hex>
sig: <BUNDLE.skb>.<digest_hex>.author.sig
```

Flags.

- --key PATH.priv (required) – PEM PKCS#8 ed25519 private key
- --identity-id ID – override the author identity (default: id:<user>@m3c)

Exit codes: 0 ok · 1 generic · 2 usage · 11 author_sig_invalid (rare; only on internal verify-after-sign self-check)

verify-sig – verify a detached author signature locally (G)

Purpose. Recompute the bundle's SHA-256 digest, locate the matching <BUNDLE.skb>.<digest_hex>.author.sig, verify it against a public key. **Local check only; does not consult the registry.** Use verify (no -sig suffix) for the full chain.

Usage.

```
skillctl verify-sig <BUNDLE.skb> --pubkey <PATH.pub>
```

Output (success).

OK: signature verified

Exit codes: 0 ok · 11 signature invalid · 1 other error · 2 usage

Trust-root management

trust list – show pinned registries (G)

Purpose. Print the parsed contents of `~/.claude/skill-trust-roots.yaml`.

Usage.

```
skillctl trust list
```

Sample output.

```
registries:
- id: aims-example-prod
  url: https://aims.example.com/api/skills
  pubkey_sha256: <hex>
  governance_minimum: green
  tenant_scope: kup-berlin
```

Exit codes: 0 ok · 2 usage

trust add – pin a registry (G)

Purpose. Add a registry to the trust-roots file.

Usage.

```
skillctl trust add --registry <url> --pubkey <path> [--id <key-id>]
```

Flags.

- `--registry URL` (required) – registry base URL (e.g. `https://aims.example.com/api/skills`)
- `--pubkey PATH` (required) – PEM SPKI ed25519 public key file
- `--id KEY-ID` – optional label (defaults to a short fingerprint-derived id)

Effect. Appends a `registries[]` entry. Idempotent on `(url, pubkey_sha256)`.

Exit codes: 0 ok · 1 generic · 2 usage

trust remove – unpin a registry (G)

```
skillctl trust remove --registry <url>
```

Removes the matching `registries[]` entry. After this, `install/verify` of any bundle whose registry was pinned at this URL will exit 12 `registry_not_pinned`.

Exit codes: 0 ok · 1 generic · 2 usage

Inventory

scan – walk skill directories (G)

Purpose. Discover all skills on the machine, classify by tier (project · user · plugin), report shadowing, optionally cross-reference with sibling `.skb` bundles for trust labels.

Usage.

```
skillctl scan [--source claude|user|plugins|all] [--format json|table] [--with-trust]
               [--root <path>] [--out <file>]
```

Flags.

- `--source` – which tier(s) to walk (default `claude`, which is `project + user + plugins` resolved as Claude Code does)
- `--format` – `table` (default for human terminals) or `json` (for scripts)
- `--with-trust` – for each skill, look for sibling `<bundle>.skb` and `<digest>.author.sig`; recompute digest and label `verified | unverified | broken`
- `--root PATH` – alternate scan root (useful for tests)
- `--out FILE` – write to file instead of stdout

Output (JSON).

```
{
  "schema": "m3c-skill-inventory/v1",
  "scanned_at": "2026-05-07T...",
  "host": "<hostname>",
  "total_count": 50,
  "by_tier": {"project": 0, "user": 14, "plugin": 36},
  "by_project": {"<repo>": 0, "<user-home>": 50},
  "skills": [
    {
      "name": "didactic-session",
      "version": "1.0.0",
      "tier": "user",
      "skill_md_sha256": "...",
      "trust": "verified",           // when --with-trust
      "bundle_digest": "sha256:...",
      "shadowed_by": null,
      "frontmatter": {...},
      "intent_status": "declared"  // or "missing", "unknown" (SPEC-0196 §7)
    }
  ]
}
```

Exit codes: 0 ok · 1 generic · 2 usage

Anchored on SKILL.md (not loose .md). A directory without `SKILL.md` is not a skill, even if it lives under `~/.claude/skills/`.

report – render a scan as HTML or markdown (G)

```
skillctl report [--format html|md] --input <scan.json> [--out <file>]
```


Useful for sharing a scan with someone who doesn't have skillctl. HTML is self-contained; markdown is GitHub-flavored.

Exit codes: 0 ok · 2 usage

diff – compare two scans (G)

```
skillctl diff <scan1.json> <scan2.json> [--output json|md|html|text] [--out <file>]
```

Reports added / removed / version-changed / digest-changed / trust-state-changed skills. Used to track drift across days or across machines.

Exit codes: 0 ok · 2 usage · non-zero if differences found (script-friendly; check stdout for the verdict)

seal – freeze a scan as baseline (G)

```
skillctl seal [--input <scan.json>] [--out <baseline.json>]
```

Writes a sealed-baseline file you can diff against later. seal runs over the most recent in-memory scan if --input omitted.

Exit codes: 0 ok · 1 generic · 2 usage

import – push a scan to a remote target (G)

```
skillctl import --target <url> [--api-key <key>] [--input <scan.json>] [--dry-run]
```

Generic upload for scan JSON. Distinct from awareness sync (which is the trust-aware admission path) and from the SPEC-0201 import-public (which fetches from upstream hubs).

Exit codes: 0 ok · 1 generic · 2 usage

audit – antivirus-shaped scanner (G) (Phase 2 implementation underway)

Purpose. Different command from scan, different default UX, different exit codes. Walks the same three tiers and labels each skill: OK | UNVERIFIED | BROKEN | BELOW_MIN.

Usage (planned interface – current build prints “audit: not yet implemented (Phase 2)” for the 1-arg form).

```
skillctl audit [--source claude|user|plugins|all] [--minimum-governance green|yellow]
               [--json] [--cleanup [--dry-run-cleanup | --confirm-delete]]
               [--allow-yellow]
```

Verdicts.

- OK – chain validates against pinned trust roots; governance ≥ minimum
- UNVERIFIED – no sibling .skb / no signature available
- BROKEN – chain fails (digest mismatch, sig invalid, registry not pinned)
- BELOW_MIN – chain validates but governance below minimum

Cleanup.

- --cleanup --dry-run-cleanup – preview which directories would be deleted

- `--cleanup --confirm-delete --apply` (deletes BROKEN and UNVERIFIED; BELOW_MIN reported but not auto-deleted – user may install with `--allow-yellow` instead)

Output footer: 47 OK · 2 unverified · 1 broken · 0 below-min.

Exit codes: 0 all OK · 2 usage · 3 at least one BROKEN · re-runs are cheap (cached by `skill_md_sha256`)

Use in CI / pre-commit:

```
skillctl audit --json --minimum-governance green || exit 1
```

review – open the local TUI for a delta (G)

```
skillctl review [--input <delta.json>] [--port 9115] [--no-browser]
```

Spawns a local browser-based viewer at `http://127.0.0.1:9115/` showing the diff output with annotation hooks. `--no-browser` for headless runs.

Exit codes: 0 ok · 2 usage

browse – read-only inventory TUI (G)

```
skillctl browse [--scan <scan.json>]
```

In-terminal browser for the inventory: list view, detail view, search. Read-only.

consolidate – fix annotation gaps in a baseline (G)

```
skillctl consolidate [--baseline <baseline.json>] [--fix]
```

Without `--fix`: reports gaps. With `--fix`: prompts and applies fixes interactively.

sync-usage – push local skill-usage telemetry (G)

Purpose. Read unsynced rows from the local SQLite database (`~/m3c-tools/skill-usage.db`) and POST each to `aims-core /api/v2/skills/usage`. Successfully synced rows are marked `synced=1`.

Usage.

```
skillctl sync-usage --target <url> [--api-key <key>]
```

Exit codes: 0 ok · 1 generic · 2 usage

Why this exists. Skill usage events are recorded locally first (offline-friendly) and pushed in batches; survives outages without losing telemetry.

Awareness bridge (SPEC-0195)

awareness sync – admit local skills to a registry (G)

Purpose. Productized version of the SKILLOR-WORK/s1 procedure. Take a `skillctl scan` inventory and admit one bundle per scanned skill via `POST /api/skills/admit-from-scan`. Bundles get `gcs_uri = local-aware://<digest>` – registry knows they exist; bytes are not transferable until republished via the Author path.

Usage.

```
skillctl awareness sync [flags]
```

Flags.

- `--source` `claude|user|plugins|all` – scan source (ignored if `--inventory` is set; default `claude`)
- `--inventory` `FILE` – read scan JSON from `FILE`; pass – to read from stdin
- `--registry` `URL` – registry URL (overrides `trust-roots default_registry` and `$M3C_REGISTRY_URL`)
- `--default-attest` `yellow|green|none` – after admission, request a default attestation (default `none`)
- `--default-intent` `yellow|green` – stamp this governance level on entries with no/UNKNOWN intent (empty = off)
- `--require-intent` – refuse to send entries with no intent or the SPEC-0196 UNKNOWN sentinel
- `--session` `TAG` – session tag (default `skill-awareness/<host>/<YYYY-MM-DD>`)
- `--dry-run` – build the envelope; do not POST
- `--confirm` – required to actually push (defense against accidental writes)
- `--key` `PATH` – author key path (PEM PKCS#8). Default `~/.claude/skillctl-keys/author.key`
- `--author-identity` `ID` – override the author identity. Default `id:<user>@m3c` (or `DevSeedSentinel` when `SKILLCTL_DEV_SEED` is set)
- `--help-advanced` – print advanced flags (e.g. `--allow-overwrite` `footgun-resistance`)

Output.

```
== skill-awareness DRY RUN ==
Backend: <registry url>
Inventory: 50 skills (36 plugin, 14 user)
Reset plan: ...
Ingest plan: ...
NO WRITES PERFORMED. Re-run without --dry-run to execute.
```

Idempotent on (digest, session_tag). Re-running with the same inventory upserts (same digest -> same document path).

Exit codes: 0 ok · 1 generic · 2 usage · 11 author_sig_invalid · 18 intent_inconsistent · 19 identity_mismatch

See also. [SKILLOR-WORK/s1/USER-MANUAL.md](#) – the original session that drove SPEC-0195’s design.

awareness verify – read back per-session admissions (G)

```
skillctl awareness verify [--session TAG] [--registry URL]
```

Reads back what was admitted under a session tag. Useful for confirming the round-trip after a sync. Default session matches the same auto-generated tag as sync (`skill-awareness/<host>/<YYYY-MM-DD>`).

Output. Table of (digest, name, version, attest_level, admitted_at).

Exit codes: 0 ok · 2 usage · 1 generic

awareness reset – delete a session’s admissions (G)

Purpose. DESTRUCTIVE. Delete only the registry docs tagged with the given session – never global, never cross-session. Two-step: dry-run produces a 5-min token; confirm-reset uses that token.

Usage.

```
# Step 1 -- dry run; produces a token
skillctl awareness reset --session TAG --dry-run
```

```
# Step 2 -- apply (within 5 min)
skillctl awareness reset --session TAG --confirm-reset --dry-run-token <sig>
```

Flags.

- `--registry URL` – registry base URL
- `--session TAG` (required) – the session whose docs to delete
- `--dry-run` – preview affected docs and produce the 5-minute token
- `--dry-run-token TOKEN` – the token from a prior `--dry-run`
- `--confirm-reset` – required (alongside `--dry-run-token`) to actually DELETE

Why two-step? The destructive blast radius is narrow by design. The dry-run token forces an explicit confirmation path; expired tokens force a re-preview.

Exit codes: 0 ok · 1 generic · 2 usage · 13 token expired (after 5 min)

Intent declaration (SPEC-0196)

intent declare – declare or update intent + data_dependencies (G)

Purpose. Update a bundle’s intent and data_dependencies declarations in the registry without re-packing. Useful for filling in missing-intent (UNKNOWN) sentinels left by awareness sync.

Usage.

```
skillctl intent declare <skill-name|@digest> [flags]
```

Flags (illustrative – run `skillctl intent declare --help` for the canonical list):

- `--side-effect TOKEN` (repeatable) – fs:write, fs:read, network:outbound, subprocess, secrets:read, llm:call, ...
- `--destructive true|false`
- `--network true|false`
- `--subprocess BINARY` (repeatable)
- `--claim CAPABILITY` (repeatable) – skill:document-generation, etc.
- `--data-dep ID:KIND:ACCESS:SCOPE:PAYLOAD_CLASS:RETENTION` (repeatable)
- `--reason FREE-TEXT` (required)
- `--key PATH.priv` (required)
- `--author-identity ID`

Cross-validation. Same rules as pack (above). Refuses with exit 18 `intent_inconsistent` on contradiction.

Exit codes: 0 ok · 1 generic · 2 usage · 18 `intent_inconsistent` · 19 `identity_mismatch`

Author ingestion (SPEC-0194)

propose – ready-to-promote gate + pack + sign + push (Y) [drafted]

Purpose. The author's daily flow. Runs the SPEC-0194 §6 ten-check ready-to-promote gate, then pack -> sign -> POST /api/skills/bundles.

Usage (planned).

```
skillctl propose <skill-name> [flags]
```

Flags.

- `--source PATH` – explicit source dir (default: resolves <skill-name> via project + user + plugins precedence)
- `--intent yellow|green|red` (required) – author's advisory governance intent
- `--rationale TEXT` (required) – what changed, why
- `--key PATH.priv` – author key (default `~/.claude/skillctl-keys/author.key`)
- `--registry URL` – overrides trust-roots default
- `--derived-from REF` – if this skill was imported via SPEC-0201, the upstream ref
- `--dry-run` – run the gate; do not pack or push

The 10 checks (SPEC-0194 §6):

1. Frontmatter complete (name, version, governance_level, owner, human_checkpoints, context_scope)
2. governance_level set
3. Smoke-test marker present (or `--allow-no-smoke` with rationale)
4. Semver bumped vs last admitted version
5. No open BUG-NNNN against this skill
6. intent declared in bundle.json
7. data_dependencies declared in bundle.json
8. intent/data_dependencies cross-validation passes
9. runtime_envelope_required set (default true for new bundles per SPEC-0202)
10. License header present (SPDX) when imported_from is set

Exit codes: 0 ok · 1 generic · 2 usage · 5 scanner_refuse · 18 intent_inconsistent / upstream_intent_capped · 19 identity_mismatch

Gate fails loud. No silent skips. Fix and re-run.

Verifier (SPEC-0188 §7, §11)

install – pull, verify, install a signed skill bundle (G)

Purpose. The end-user's primary command. Pulls a bundle by name@version (or digest), runs the full chain check, resolves depends_on, atomically extracts to `~/.claude/skills/<name>/`. **Any failure = no write.**

Usage.

```
skillctl install <name>[@<version>] [flags]
```

Flags.

- `--registry URL` – registry base URL (required only when trust-roots has multiple registries pinned)
- `--governance-min green|yellow` – override the trust-root's `governance_minimum` (empty = use trust-root)
- `--allow-yellow` – lower the gate from green to yellow for this install (audited)
- `--ignore-deps` – skip `depends_on` resolution (audited)
- `--tenant ID` – pin this install to a tenant scope (SPEC-0188 §7 step 5.5). Overrides trust-roots `tenant_scope`. Empty = use trust-roots value or run untenanted.
- `--verbose` – print structured per-step log lines to stderr
- `--home PATH` – override the install root (advanced; defaults to `$HOME`)

Chain check order (SPEC-0188 §7):

1. Resolve digest via `/api/skills/by-name`
2. Recompute digest from fetched bytes; compare -> exit 10 on mismatch
3. Verify `<digest>.author.sig` against the registered author identity -> exit 11 on fail
4. Verify `<digest>.registry.sig` against pinned trust roots -> exit 12 on fail
5. Read `<digest>.governance.sig`; check governance level $\geq$ minimum -> exit 13 on fail 5.5. Apply tenant policy -> exit 13 (block) or exit 16 (revoked) as applicable
6. Check all `data_dependencies` are authorized for the tenant -> exit 14 on any denied
7. Resolve `depends_on` recursively -> exit 15 on any unsatisfied
8. Atomic move to `~/.claude/skills/<name>/`

Exit codes: 0 ok · 1 generic · 2 usage · 10..16 per chain check (see [USER-MANUAL.md §8.2](#))

Reverse the install. Either `rm -rf ~/.claude/skills/<name>/` (then `skillctl audit` will not list it) or use the planned `skillctl uninstall` (queued).

verify – re-run the trust-chain check on an installed skill (G)

Purpose. Same chain check as `install`, but against an already-installed skill (no re-fetch). Use after `audit` finds something suspicious.

Usage.

```
skillctl verify <name> [flags]
```

Flags.

- `--registry URL` – same as `install`
- `--governance-min green|yellow`
- `--allow-yellow` – permit yellow result against a green-required trust root (does NOT re-audit; per-call)
- `--tenant ID` – pin this verify to a tenant scope
- `--verbose`
- `--home PATH`

Exit codes: identical to `install` (0, 2, 10..16).

Governance attestation

attest – POST a signed governance attestation (G)

Purpose. Reviewer signs (`bundle_digest`, `governance_level`, `review_timestamp`, `reviewer_identity`) and posts to `_skill attestations`. Multiple attestations per bundle allowed; the **most recent** by a reviewer with the required role is the binding verdict.

Usage.

```
skillctl attest <bundle-digest> \  
  --level green|yellow|red \  
  --rationale TEXT \  
  --reviewer-id ID \  
  --key PATH.priv \  
  [--registry URL]
```

Flags.

- <bundle-digest> (positional, required) – sha256:<hex>
- --level green|yellow|red (required) – governance verdict
- --rationale TEXT (required) – free-text rationale (audit metadata; **NOT** folded into signed bytes)
- --reviewer-id ID (required) – reviewer identity (e.g. id:reviewer@m3c)
- --key PATH.priv (required) – PEM PKCS#8 ed25519 private key (mode 0600)
- --registry URL – registry base URL (default: https://aims-core/api/skills or env)

v1 limitation. In registry-on-behalf-of mode (CEO briefing Q3 lock) the registry key signs the attestation; the audit log records the human reviewer, but the byte-level signature is the registry's. Per-reviewer keys land Q3-2026.

Exit codes: 0 ok · 1 generic · 2 usage · 11 author_sig_invalid (the signed payload couldn't be re-verified server-side) · 19 identity_mismatch (reviewer-id not registered)

Upstream airlock (SPEC-0201)

import-public – stage an upstream bundle (Y) [drafted]

Purpose. Fetch a skill from a public hub, place it under ~/.cache/m3c/imports/<sha256>/, run static scanners, write scan-report.json. **Does NOT install or admit** – staging only.

Usage.

```
skillctl import-public <scheme>://<ref>[@sha256:<hex>] [flags]
```

Schemes. skillhub, claudeskills, lobehub, skillforge, tonsofskills, skillsmp, gh (GitHub) – gated by ~/.claude/skill-import-policy.yaml.

Two-step pinning. First call without @sha256:... prints the required pin and refuses (exit 4). Second call with the pin stages the bytes.

Flags.

- --accept-license SPDX (required) – explicit license acknowledgement
- --dry-run – fetch + scan; do not stage
- --policy PATH – override ~/.claude/skill-import-policy.yaml

Refuses on: secret regex hits, os.system / shell-true, file writes outside skill dir, HTTP to non-allowlisted hosts, missing license header, CVE-high deps.

Exit codes: 0 ok · 2 usage · 4 pin_required · 5 scanner_refuse · 17 no_source_policy · 19 source_blocked

import-list – list staged imports (Y) [drafted]

```
skillctl import-list [--json]
```

Lists everything under `~/.cache/m3c/imports/`. Per row: source ref, sha256, license, scanner verdict, age.

import-policy lint – validate the source policy (Y) [drafted]

```
skillctl import-policy lint <path>
```

Validates `~/.claude/skill-import-policy.yaml` against the SPEC-0201 schema. Reports unknown keys, missing required fields, conflicting allow/deny rules.

Exit codes: 0 valid · 1 invalid (printed) · 2 usage

Runtime envelope (SPEC-0202)

run – wrap an invocation under a capability token (Y) [drafted]

Purpose. Issue a short-lived signed capability token, set `M3C_SKILL_CAPABILITY_TOKEN` in the child env, inject the gateway shim via `LD_PRELOAD/DYLD_INSERT_LIBRARIES`, exec the skill. Every HTTP/subprocess/file call is gated; refusals are audited.

Usage.

```
skillctl run <name> [flags]
```

Flags.

- `--tenant ID` – tenant scope (intersected with caller policy)
- `--max-runtime SECONDS` – request a larger envelope (intersected by tenant policy)
- `--max-egress-bytes N`
- `--capability TOKEN` (repeatable) – request additional capabilities (e.g. `subprocess_run:bash`); intersected with bundle declaration and tenant policy
- `--dry-issue` – issue the token; print the envelope; do not exec

Envelope = (bundle declaration) ∩ (tenant policy) ∩ (caller request). Empty intersection -> no token issued, exit 1 with 403 `envelope_empty`.

Defenses against bypass (SPEC-0202):

1. **Bundle declaration** – `runtime_envelope_required: true` (default for new bundles); reviewer attesting (G) verifies the skill imports the gateway; `skillctl pack` lints it.
2. **skillctl run wrapper** – `LD_PRELOAD/DYLD_INSERT_LIBRARIES` injects the gateway shim into the process tree.
3. **SOCKS egress sentinel** – per-tenant outbound network steered through a SOCKS proxy that requires the active `token_id` as auth.

A skill cannot escape all three without the consumer machine being fundamentally compromised.

Exit codes: runtime gate violations surface verbatim – 30 `envelope_capability_missing` · 31 `envelope_data_source_missing` · 32 `egress_host_not_allowed` · 33 `subprocess_denied` · 34 `file_outside_envelope` · 35 `token_expired` · 36 `token_revoked` · 37 `token_invalid_signature` · 38 `runtime_quota_exceeded` · 39 `egress_byte_quota_exceeded` · plus 403 `governance_below_minimum`, 403 `envelope_empty`, 403 `source_blocked`, 503 `issuer_overloaded` from the issuer.

invoke-replay – re-run a recorded invocation (Y) [drafted]

Purpose. The Kafka event log makes invocations deterministic. Replay against either the recorded policy (`--mode dry`, sanity check) or a new tighter policy (`--mode live`, pen-test).

Usage.

```
skillctl invoke-replay <token-id> --mode dry|live [--tenant ID] [--policy PATH]
```

Use cases.

- Validate that a policy change actually closes a class of behavior.
- Demonstrate to an auditor that a past invocation was within envelope.
- Pen-test a new policy against historical traffic before publishing.

Output. Sequence of `gate.allowed` / `gate.refused` events, plus a divergence report when `--mode live` differs from the recording.

Author-key revocation (SPEC-0198)

There is no top-level subcommand for this in the dispatcher today. The flow runs through the registry API:

```
curl -X POST $REG/api/skills/identities/<id>/revoke \
  -H "Authorization: Bearer $ADMIN_TOKEN" \
  -d '{"effective_at": "<ISO>", "rationale": "<why>"}'
```

A future `skillctl identity revoke <id>` is queued (SPEC-0198 §X). The bash form above is the canonical v1 flow.

Conventions in command output

- **Stderr** – all diagnostics, all error messages, structured `--verbose` lines.
 - **Stdout** – machine-readable result. JSON when `--format json` or `--json`. Bare value (digest, fingerprint) for single-result calls.
 - **Exit code** – branch on this. The numbered codes are stable; log strings are not.
 - **No interactive prompts** by default. Destructive commands (`audit --cleanup`, `awareness reset`) require explicit confirmation flags.
-

Notable cross-command flags

Flag	Commands	Meaning
<code>--registry URL</code>	<code>awareness</code> , <code>attest</code> , <code>install</code> , <code>verify</code> , <code>intent</code> , <code>propose</code> , <code>import-*</code>	Override the registry endpoint
<code>--key PATH.priv</code>	<code>sign</code> , <code>attest</code> , <code>awareness sync</code> , <code>intent</code> , <code>propose</code>	Path to ed25519 private key (mode 0600)
<code>--tenant ID</code>	<code>install</code> , <code>verify</code> , <code>run</code> , <code>invoke-replay</code>	Pin operation to a tenant scope
<code>--allow-yellow</code>	<code>install</code> , <code>verify</code>	Lower the governance gate per-call (audited)

Flag	Commands	Meaning
<code>--dry-run</code>	awareness sync/reset, import, import-public, propose	Preview without applying
<code>--verbose</code>	install, verify, run	Per-step structured stderr logs
<code>--json</code>	scan, audit, import-list, intent declare	Machine-readable output
<code>--home PATH</code>	install, verify	Override \$HOME for tests

Worked examples (smallest viable runs)

E1 – Sign and verify a bundle locally (no registry)

```
# 1) Generate a keypair
skillctl keygen --out /tmp/tmp-key

# 2) Build a deterministic .skb
skillctl pack ~/.claude/skills/hello-skill --out /tmp/hello-0.1.0.skb

# 3) Sign it
skillctl sign /tmp/hello-0.1.0.skb --key /tmp/tmp-key.priv --identity-id id:test@local

# 4) Verify locally (no chain, just signature)
skillctl verify-sig /tmp/hello-0.1.0.skb --pubkey /tmp/tmp-key.pub
# -> "OK: signature verified"
```

E2 – Pin a registry, install, run

```
# 1) Pin
skillctl trust add \
  --registry https://aims.example.com/api/skills \
  --pubkey ~/.config/m3c/skill-trust-roots/aims-prod.pub \
  --id aims-prod

# 2) Install (full chain check)
skillctl install didactic-session@1.0.0 --tenant kup-berlin

# 3) Run (envelope-bounded)
skillctl run didactic-session --tenant kup-berlin
```

E3 – SKILLOR-style awareness sync

```
# 1) Inventory
skillctl scan --source claude --with-trust --format json > /tmp/inv.json

# 2) Dry-run
skillctl awareness sync \
  --inventory /tmp/inv.json \
  --registry https://aims.example.com/api/skills \
```

```

--default-attest yellow \
--session "skill-awareness/${hostname -s}/${date +%F}" \
--dry-run

# 3) Confirm
skillctl awareness sync \
  --inventory /tmp/inv.json \
  --registry https://aims.example.com/api/skills \
  --default-attest yellow \
  --session "skill-awareness/${hostname -s}/${date +%F}" \
  --confirm

# 4) Verify round-trip
skillctl awareness verify --session "skill-awareness/${hostname -s}/${date +%F}"

```

E4 – Reviewer attestation

```

DIGEST="sha256:cd1d5c79..."
skillctl attest "$DIGEST" \
  --level green \
  --rationale "Activation gate passed; intent matches data_deps; smoke-tested locally." \
  --reviewer-id id:reviewer@m3c \
  --key ~/.claude/skillctl-keys/reviewer.key \
  --registry https://aims.example.com/api/skills

```

E5 – Import from a public hub (drafted SPEC-0201)

```

# 1) Without the pin -- refuses, prints required sha256
skillctl import-public skillhub://didactic-session
# -> "exit 4 pin_required: re-run with @sha256:abc123..."

# 2) With the pin
skillctl import-public skillhub://didactic-session@sha256:abc123... \
  --accept-license MIT

# 3) Inspect
skillctl import-list
cat ~/.cache/m3c/imports/abc123.../scan-report.json

# 4) Promote to candidate
skillctl propose didactic-session \
  --source ~/.cache/m3c/imports/abc123.../ \
  --derived-from skillhub://didactic-session@sha256:abc123... \
  --intent yellow \
  --rationale "imported from skillhub; pending reviewer"

```

What this manual does NOT cover

- **Server-side admin endpoints** (/api/skills/identities, /api/tenants/*, /api/skills/audit-log). See `aims-core/flask/modules/skill_registry/api.py` and the role playbook in [USER-MANUAL.md §5.4 / §5.5](#).
 - **The CISO Console UI** (/v2/ciso?tenant=<id>). See [USER-MANUAL.md §5.3](#) and SPEC-0192.
 - **The MCP skill-server tools** (`skill_install`, `skill_verify`, `skill_attest`, `skill_scan` for use directly from Claude Code). See SPEC-0122 and `m3c-tools/mcp-skill-server`.
 - **The KafScale fan-out interface** (interface frozen in SPEC-0193; CLI surface lands once KafScale itself ships).
 - **The federation protocol** for cross-pool republication (SPEC-0197 – Silver/Gold tier feature; not on the v1 single-pool path).
-

References

- [USER-MANUAL.md](#) – role-keyed playbooks, exit-code catalog, decision tree, troubleshooting
- [SKILLOR-WORK/s1/USER-MANUAL.md](#) – the original awareness session that the productized commands derive from
- [CONCEPT.md](#) – production architecture, semantic-graph proposition, productization
- [ROLES.md](#) – roles, interaction patterns, conflict zones
- [IMPORT-AIRLOCK.md](#) – operator brief for SPEC-0201
- [RUNTIME-ENVELOPE.md](#) – operator brief for SPEC-0202
- SPECS: [-0188](#) · [-0189](#) · [-0194](#) · [-0195](#) · [-0196](#) · [-0198](#) · [-0201](#) · [-0202](#)
- Source of truth: `cmd/skillctl/main.go` on `s2/integration` (worktree `/Users/kamir/wt/spec-0189/s2-integration/`)